Pocket Packet Sniffer

Software Requirements Specification

WiFi / Ethernet / BLE Portable Packet Sniffing & Dashboard Tool

Document ID	PPS-SRS-1.0
Version	1.0
Standard	IEEE Recommended Practice Format
Date	28 September 2026
Status	Draft for Review — Part 1 Deliverable

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the Pocket Packet Sniffer (PPS), a portable, battery-powered network diagnostic and security monitoring device. This document is written in accordance with the recommended practice for software requirements specifications (structure modeled on IEEE 830 / IEEE 29148) and is intended to serve as the authoritative basis for design, implementation, and acceptance testing of Part 1 of the mini-project.

1.2 Document Conventions

Requirements are uniquely identified using the prefixes FR-n (Functional Requirement), NFR-<category>-n (Non-Functional Requirement) and SEC-OBJ-n / SEC-REQ-n (Security Objective / Security Requirement). The keyword “shall” denotes a mandatory, verifiable requirement; “should” denotes a recommended but non-mandatory capability. Every requirement in Sections 4 and 5 is written to be clear (single interpretation), unambiguous (no vague qualifiers such as “fast” or “user-friendly” without a measurable bound), concise (one requirement per statement), verifiable (testable via an objective pass/fail criterion), and traceable (linked to a unique ID referenced in the Test Plan and Design Specification).

1.3 Intended Audience and Reading Suggestions

- Project developers and student team members implementing the Pi/ESP32/Python/Zeek stack.
- Course instructor / evaluator assessing Part 1 deliverables.
- Future maintainers extending capture protocols or dashboard features.

Readers should review Section 2 for context before consulting the detailed requirements in Sections 4 and 5.

1.4 Product Scope

The Pocket Packet Sniffer is a handheld, battery-operated device (Raspberry Pi + ESP32-based capture front end) that passively captures WiFi (802.11), wired Ethernet, and Bluetooth Low Energy (BLE) traffic, analyzes it in real time using the Zeek network security monitor, and presents live statistics and security alerts on a self-hosted portable web dashboard. It is intended for field network diagnostics, WiFi/BLE site surveys, and lightweight on-site intrusion/anomaly detection where connecting a laptop or cloud SIEM is impractical. The device is self-contained: it does not require internet connectivity to operate, serving its dashboard over a local WiFi access point.

1.5 References

- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.
- IEEE Std 29148-2018, Systems and Software Engineering – Life Cycle Processes – Requirements Engineering.
- IEEE Std 829-2008, Standard for Software and System Test Documentation.
- Zeek Network Security Monitor Documentation, zeek.org.
- Scapy / PyShark packet manipulation library documentation.
- Software Architecture & Design Specification – Pocket Packet Sniffer (companion document, PPS-SADS-1.0).
- Test Plan – Pocket Packet Sniffer (companion document, PPS-TP-1.0).

2. Overall Description

2.1 Product Perspective

The Pocket Packet Sniffer is a new, self-contained embedded product; it is not a component of a larger pre-existing system. It combines commodity single-board-computer hardware (Raspberry Pi) with a microcontroller-based radio front end (ESP32) for WiFi promiscuous-mode and BLE capture, a wired Ethernet tap, the open-source Zeek analysis engine, and a Python-based backend/dashboard. Figure 1 (Section 6) shows the primary actors and use cases; the companion Software Architecture & Design Specification details the internal component structure.

2.2 Product Functions

- Multi-protocol packet capture: WiFi (monitor mode), wired Ethernet, and BLE advertisements/GATT traffic.
- Real-time traffic analysis and anomaly/intrusion detection via Zeek scripts and a custom rule engine.
- Portable, locally hosted web dashboard for live visualization, alerting, and session control.
- Secure local storage and export of raw captures (.pcap) and summary reports.
- User authentication, role-based access, and audit logging for accountability.

2.3 User Classes and Characteristics

User Class	Description	Technical Expertise
Field Engineer	Primary operator; deploys the device on-site, starts/stops captures, reviews dashboard, exports evidence.	Moderate – familiar with networking concepts, not necessarily a developer.
System Administrator	Configures the device, manages user accounts, storage retention, and firmware/software updates.	High – comfortable with Linux administration and security configuration.
Security Analyst (secondary)	Consumes exported .pcap/reports offline in tools such as Wireshark for deeper forensic analysis.	High – network security background.

2.4 Operating Environment

- Hardware: Raspberry Pi 4B / Zero 2W (host), ESP32-WROOM module(s) for WiFi/BLE capture, USB/onboard Ethernet NIC, microSD or USB storage, Li-ion battery pack.
- Operating System: Raspberry Pi OS (Debian-based, 64-bit) for the host; FreeRTOS/Arduino core for the ESP32 firmware.
- Software: Python 3.10+, Zeek 6.x, Flask/FastAPI, Scapy/PyShark, SQLite; dashboard rendered in any modern mobile/desktop browser over local WiFi.
- Deployment: Field environments (offices, factory floors, outdoor sites) with ambient temperatures of 0–40°C and no guaranteed internet connectivity.

2.5 Design and Implementation Constraints

- Must run entirely on-device with no mandatory cloud dependency (offline-first).
- Limited onboard compute/power budget of a Raspberry Pi and battery pack constrains capture throughput and Zeek ruleset complexity.
- Must use open-source, license-compatible components (Zeek is BSD-licensed; Scapy is GPLv2).
- Local storage capacity (typically 32–128 GB microSD/USB) bounds retained capture history.
- WiFi monitor-mode capture depends on chipset/driver support of the ESP32/host WiFi adapter.

2.6 Assumptions and Dependencies

- It is assumed the operator has legal authorization to capture traffic in the deployment environment.

- The system depends on the continued availability and compatibility of the Zeek open-source project.
- It is assumed a modern browser (Chrome, Firefox, Safari) is available on the client device used to view the dashboard.
- Python and required libraries are installed.
- The selected hardware supports the required packet capture functionality.

3. External Interface Requirements

3.1 User Interfaces

The primary interface is a responsive, locally hosted single-page web dashboard (HTML5/JavaScript) accessed by connecting a phone/laptop to the device's WiFi hotspot and navigating to its local IP address. The dashboard exposes capture controls, live traffic graphs, an alert panel, and an export/report screen. An optional small on-device OLED display shows boot status, IP address, and battery level.

3.2 Hardware Interfaces

- USB / GPIO UART interface between Raspberry Pi and ESP32 capture module(s).
- Onboard/USB Gigabit or 100 Mbps Ethernet NIC for wired capture.
- microSD card slot and/or USB 3.0 port for storage expansion.
- Battery management circuit with I2C fuel-gauge interface for power/battery telemetry.

3.3 Software Interfaces

- Zeek engine invoked via its command-line/Broker interface; log files consumed from the local filesystem.
- Scapy/PyShark Python libraries for packet parsing and .pcap read/write.
- SQLite database for session metadata, alerts, users, and audit logs.
- REST/JSON and WebSocket APIs between backend and dashboard front end (see SADS Section 3.2 for full API design).

3.4 Communications Interfaces

- Local WiFi access point (802.11n/ac) hosting the dashboard, secured with WPA2/WPA3.
- All dashboard/API traffic transported over HTTPS/WSS (TLS 1.2+) even on the local link.
- Optional Bluetooth/serial link for firmware updates to the ESP32 capture module.

4. System Features — Functional Requirements

Each functional requirement below is atomic, uses a mandatory “shall” statement, includes a measurable acceptance bound where applicable, and is uniquely identified for traceability to the Test Plan (Section 5.1) and the Software Architecture & Design Specification (Section 4).

ID	Feature Group	Requirement Statement
FR-1	Multi-Interface Capture	The system shall capture 802.11 WiFi frames in monitor mode on a user-selected 2.4/5 GHz channel.
FR-2	Multi-Interface Capture	The system shall capture Ethernet frames from the wired interface at line rates of up to 100 Mbps with no configuration beyond interface selection.
FR-3	Multi-Interface Capture	The system shall capture BLE advertisement and GATT packets within a configurable RSSI threshold set by the user.
FR-4	Capture Configuration	The system shall allow the user to define a capture filter (protocol, IP, port, MAC/SSID) before starting a session, using BPF-compatible syntax.
FR-5	Analysis	The system shall route captured traffic through the Zeek analysis engine and produce structured logs (conn.log, dns.log, weird.log, notice.log) within 2 seconds of packet capture.
FR-6	Dashboard	The system shall display live capture statistics (packets/sec, top talkers, protocol distribution) on the dashboard, refreshed at an interval of 2 seconds or less.
FR-7	Session Control	The system shall allow an authenticated user to start and stop a capture session from the dashboard UI.
FR-8	Export	The system shall export the current or a stored capture session as a standards-compliant .pcap/.pcapng file on user request.
FR-9	Reporting	The system shall generate a human-readable session summary report (HTML or PDF) containing capture duration, packet/byte counts, and a list of detected anomalies.
FR-10	Anomaly Detection	The system shall detect and flag, at minimum, ARP spoofing, TCP/UDP port-scan patterns, rogue access points (SSID/BSSID mismatch), and BLE advertisement flooding.
FR-11	Alerting	The system shall raise a security alert, visible on the dashboard within 5 seconds of detection, whenever an anomaly rule is triggered.
FR-12	Alerting	The system shall allow an authenticated user to acknowledge and clear an active alert, recording the acknowledging user and timestamp.
FR-13	Administration	The system shall allow a user with the Administrator role to view storage usage, delete stored capture sessions, and configure a retention period (in days).
FR-14	Connectivity	The system shall operate as a local WiFi access point so that the dashboard is reachable by a client device without any external internet connection.
FR-15	Concurrent Capture	The system shall support at least two simultaneously active capture interfaces (e.g., WiFi + Ethernet) with a combined packet drop rate of no more than 1% at the reference load defined in the Test Plan.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

ID	Requirement Statement
NFR-PERF-1	The dashboard shall reflect newly arrived backend statistics within 2 seconds, sustained under a load of up to 5,000 packets/second.
NFR-PERF-2	The system shall sustain continuous capture for at least 4 hours on battery power without data loss, on the reference hardware configuration defined in Section 2.4.
NFR-PERF-3	The system shall boot and reach an operational dashboard state within 90 seconds of power-on.

5.2 Reliability & Availability Requirements

ID	Requirement Statement
NFR-REL-1	The system shall not lose more than 0.1% of captured packets during any single continuous capture session of up to 30 minutes at 1,000 packets/second.
NFR-REL-2	The system shall automatically resume dashboard availability within 30 seconds of an unexpected backend process crash, via a supervised restart mechanism.

5.3 Usability & Portability Requirements

ID	Requirement Statement
NFR-USE-1	A first-time user shall be able to start a basic WiFi capture session within 3 minutes of powering on the device, using only the on-device quick-start guide.
NFR-PORT-1	The dashboard shall render correctly (no layout breakage) on mobile browser viewports as narrow as 360 px wide.

5.4 Maintainability Requirements

ID	Requirement Statement
NFR-MAINT-1	All backend Python modules shall achieve at least 70% unit-test statement coverage, verified by automated CI on every merge.
NFR-MAINT-2	The system shall log all unhandled backend exceptions with a stack trace to a persistent log file for post-hoc debugging.

5.5 Security Section

This section defines the security objectives that scope the system's threat mitigation goals and the corresponding verifiable security requirements. It is elaborated further in the Security Architecture (SADS Section 2.5) and validated per the Security Validation Approach (Test Plan Section 5.1).

5.5.1 Security Objectives

ID	Objective	Description
SEC-OBJ-1	Confidentiality	Protect the confidentiality of captured traffic data and derived analysis artifacts — which may contain sensitive payloads, credentials, or PII — both at rest on device storage and in transit to the dashboard/API.
SEC-OBJ-2	Access Control	Ensure that only authenticated, authorized personnel can control captures, view live traffic, or retrieve/export stored data and system configuration.

ID	Objective	Description
SEC-OBJ-3	Integrity & Accountability	Preserve the integrity of captured evidence and maintain a tamper-evident, non-repudiable audit trail of security-relevant actions for forensic and compliance purposes.

5.5.2 Security Requirements

ID	Requirement Statement
SEC-REQ-1	The system shall encrypt all locally stored .pcap files and the session/metadata database at rest using AES-256.
SEC-REQ-2	The system shall require authentication (username and password of at least 10 characters, or an equivalent public-key/token credential) before granting access to the dashboard or any API endpoint.
SEC-REQ-3	The system shall enforce Role-Based Access Control with, at minimum, a Field Engineer role (capture and view) and an Administrator role (capture, view, configure, and manage users).
SEC-REQ-4	The system shall encrypt all communication between the dashboard UI, backend API, and any client device using TLS 1.2 or higher.
SEC-REQ-5	The system shall maintain an append-only, hash-chained audit log of all authentication events, configuration changes, and data export actions, tamper-evident against post-hoc modification.
SEC-REQ-6	The system shall lock an account for 5 minutes after 5 consecutive failed login attempts and shall log each lockout event.

6. UML Use Case Diagram

Figure 1 illustrates the primary actors — Field Engineer, System Administrator, and the Zeek Engine (modeled as a supporting actor representing the analysis subsystem) — and the principal use cases of the Pocket Packet Sniffer, including include-relationships between capture/filter configuration and downstream anomaly detection and alerting.

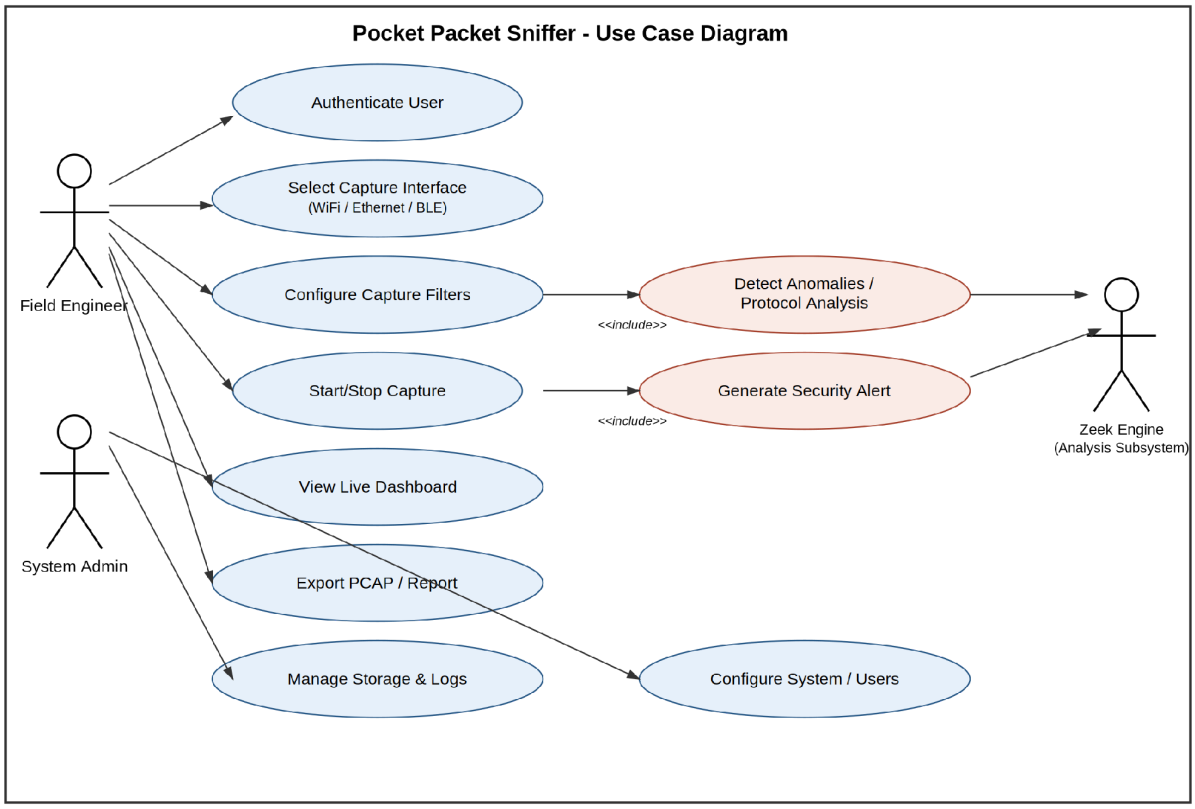


Figure 1. Pocket Packet Sniffer — UML Use Case Diagram

6.1 Use Case Traceability Summary

Use Case	Primary Actor	Related Requirement(s)
Authenticate User	Field Engineer, Admin	SEC-REQ-2, SEC-REQ-6
Select Capture Interface	Field Engineer	FR-1, FR-2, FR-3, FR-15
Configure Capture Filters	Field Engineer	FR-4
Start/Stop Capture	Field Engineer	FR-7
Detect Anomalies / Protocol Analysis	Zeek Engine	FR-5, FR-10
Generate Security Alert	Zeek Engine	FR-11
View Live Dashboard	Field Engineer	FR-6
Export PCAP / Report	Field Engineer	FR-8, FR-9
Manage Storage & Logs	Admin	FR-13
Configure System / Users	Admin	SEC-REQ-3

7. Appendix

7.1 Glossary

Term	Definition
BLE	Bluetooth Low Energy – a low-power wireless protocol used for short-range device communication.
BPF	Berkeley Packet Filter – a filter expression syntax used to select packets during capture.
Zeek	An open-source network security monitor that converts raw traffic into high-level, structured event logs.
RBAC	Role-Based Access Control – an access control model where permissions are assigned to roles, and roles to users.
pcap/pcapng	Packet capture file formats used to store raw network traffic for later analysis.
RSSI	Received Signal Strength Indicator – a measurement of the power present in a received radio signal.

7.2 Requirements Traceability Note

All requirement IDs defined in Sections 4 and 5 are carried forward unchanged into the companion Test Plan (traceability matrix, Section 6) and Software Architecture & Design Specification (traceability tables, Sections 2.4 and 4), ensuring end-to-end coverage from requirement to design to test case.